

BAC PRO CIEL
TRAVAUX PRATIQUES — PROGRAMMATION

GESTIONNAIRE DE MOTS DE PASSE — IMPLÉMENTATION COMPLÈTE

Nom : _____
Prénom : _____
Classe : _____

Durée : 16 heures
Date : _____
Note : _____ / 20

OBJECTIF DE CE TP

Vous allez programmer **ENTIÈREMENT** le gestionnaire de mots de passe sur Arduino.
Le circuit est déjà câblé et validé (Séances 1 à 3).

À la fin de ce TP, votre programme doit :

1. Demander une clé secrète au démarrage pour s'authentifier
2. Proposer un menu (Stocker / Lire / Supprimer / Lister)
3. Chiffrer les mots de passe avec l'algorithme XOR avant stockage
4. Déchiffrer à la demande et afficher sur le LCD
5. Donner un retour visuel (LEDs) et sonore (buzzer) à chaque action

Vous construirez le programme **BLOC PAR BLOC** dans l'ordre du TP.
Chaque bloc est testé et validé avant de passer au suivant.

MÉTHODE DE TRAVAIL OBLIGATOIRE

Lisez **ENTIÈREMENT** chaque section avant d'écrire le moindre code.
Compilez et testez après chaque bloc — ne jamais écrire 100 lignes sans tester.
En cas d'erreur de compilation : lisez le message d'erreur mot par mot.
Ne passez **JAMAIS** à l'étape suivante sans la validation du professeur.

PLAN DU TP	Durée estimée
BLOC 1 — Structure du programme et déclarations	25 min
BLOC 2 — Saisie d'une clé via le clavier (fonction saisirTexte)	20 min
BLOC 3 — Chiffrement XOR (fonction chiffrerXOR)	30 min
BLOC 4 — Stocker un mot de passe (fonction stockerMdp)	35 min
BLOC 5 — Lire un mot de passe (fonction lireMdp)	25 min
BLOC 6 — Supprimer un mot de passe (fonction supprimerMdp)	25 min
BLOC 7 — Afficher la liste (fonction afficherListe)	15 min
BLOC 8 — Authentification au démarrage	20 min
BLOC 9 — Le menu principal dans loop()	20 min
BLOC 10 — Tests complets et validation finale	25 min
Guide débogage — Erreurs fréquentes et solutions	Référence
BLOC 11 (BONUS) — Persistance EEPROM	30 min

CARTE DU PROGRAMME — VUE D'ENSEMBLE

Avant d'écrire une seule ligne, lisez et comprenez la structure complète du programme.



POINT CLÉ — POURQUOI COPIER DANS bufTemp ?

coffre[idx].mdpChiffre contient TOUJOURS la version chiffrée.

Si on appelle chiffrerXOR directement dessus, il deviendrait lisible en clair dans le tableau.

La 2ème lecture du même mot de passe serait alors corrompue (XOR d'un texte déjà clair).

La copie dans bufTemp protège l'intégrité du coffre.

BLOC 1 — STRUCTURE DU PROGRAMME ET DÉCLARATIONS

 25 minutes

CE QUE CONTIENT UN FICHIER ARDUINO .ino

Un programme Arduino est organisé en 3 zones dans cet ordre précis :

ZONE A — En-tête : les #include (bibliothèques) et toutes les déclarations globales

ZONE B — setup() : code exécuté UNE SEULE FOIS au démarrage

ZONE C — loop() : code exécuté en BOUCLE INFINIE

ZONE D — Fonctions personnalisées : écrites après loop(), appelées depuis setup/loop

Règle : toute variable déclarée dans l'en-tête est accessible PARTOUT dans le programme.

Une variable déclarée dans une fonction n'existe QUE dans cette fonction.

1.1 — Bibliothèques nécessaires

Chaque #include importe une bibliothèque. Copiez ces lignes en tout premier dans votre fichier .ino.

```
// =====  
// GESTIONNAIRE DE MOTS DE PASSE — BAC PRO CIEL  
// Auteur : _____ Classe : _____  
// Date   : _____  
// =====  
  
#include <Keypad.h>           // gestion du clavier 4x4  
#include <Wire.h>             // protocole I2C (requis par le LCD)  
#include <LiquidCrystal_I2C.h> // pilote de l'écran LCD I2C
```

QUESTION 1.1

Pourquoi faut-il inclure Wire.h en plus de LiquidCrystal_I2C.h alors qu'on ne fait pas appel à Wire directement dans notre code ? Quel est le rôle de Wire.h dans la communication avec le LCD ?

1.2 — Configuration du clavier 4x4

Le clavier matriciel a besoin de 3 éléments : la disposition des touches, les pins de lignes et les pins de colonnes.

```
// — CLAVIER MATRICIEL 4x4 —  
const byte LIGNES = 4; // nombre de lignes  
const byte COLONNES = 4; // nombre de colonnes  
  
// Disposition des touches (ligne par ligne, de haut en bas)  
char touches[LIGNES][COLONNES] = {  
    {'1','2','3','A'}, // ligne 1 — rangée du haut
```

```

    {'4','5','6','B'},    // ligne 2
    {'7','8','9','C'},    // ligne 3
    {'*','0','#','D'}     // ligne 4 – rangée du bas
};

// Pins Arduino connectés aux LIGNES du clavier (R1 à R4)
byte pinsLignes[LIGNES] = {9, 8, 7, 6};
// Pins Arduino connectés aux COLONNES du clavier (C1 à C4)
byte pinsColonnes[COLONNES] = {5, 4, 3, 2};

// Création de l'objet clavier avec ces paramètres
Keypad clavier = Keypad(makeKeymap(touches), pinsLignes, pinsColonnes, LIGNES,
COLONNES);

```

QUESTION 1.2

Dans le tableau `touches[LIGNES][COLONNES]`, la touche '7' se trouve à la ligne 2 (index 2), colonne 0 (index 0). Si un élève appuie sur '7', l'Arduino détecte que la ligne 2 (pin 7) et la colonne 0 (pin 5) sont en contact. Expliquez en 3 phrases le mécanisme de balayage matriciel qui permet à l'Arduino d'identifier quelle touche précise a été pressée.

1.3 — Constantes des pins et LCD

```

// — PINS DES LEDS ET DU BUZZER —
const int LED_VERT = 13;    // allumée = mot de passe trouvé / action OK
const int LED_BLEUE = 12;   // allumée = saisie en cours
const int LED_JAUNE = 11;   // allumée = suppression effectuée
const int BUZZER     = 10;   // bip court = OK / bip long = ERREUR

// — ÉCRAN LCD —
// Adresse I2C : 0x27 (vérifiez avec le scanner I2C si le LCD ne s'allume pas)
// 16 colonnes, 2 lignes
LiquidCrystal_I2C lcd(0x27, 16, 2);

```

SI VOTRE LCD NE S'ALLUME PAS

L'adresse I2C 0x27 peut varier selon le fabricant du module. Si l'écran reste noir :

1. Téléchargez et lancez le sketch 'I2C Scanner' depuis les exemples Arduino
2. Il affichera l'adresse réelle dans le Serial Monitor
3. Remplacez 0x27 par l'adresse trouvée (souvent 0x3F)

1.4 — Structure de données et variables globales

La structure `MotDePasse` regroupe les informations d'une entrée du coffre. Le tableau `coffre[]` stockera toutes les entrées.

```
// — STRUCTURE DE DONNÉES —
// Un MotDePasse regroupe : le nom du site + le mot de passe chiffré
struct MotDePasse {
    char site[20];          // nom du site (ex : 'GMAIL', max 19 caractères +
                           // terminateur)
    char mdpChiffre[50];    // mot de passe chiffré par XOR (max 49 caractères +
                           // terminateur)
};

// — VARIABLES GLOBALES —
const int MAX_MDP = 5;          // capacité maximale du coffre
MotDePasse coffre[MAX_MDP];    // tableau qui stocke tous les mots de passe
int      nbMdp      = 0;        // nombre de mots de passe actuellement
stockés
int      nbEchecs   = 0;        // compteur d'échecs de connexion
char     cleSecrete[20] = "CIEL2025"; // clé XOR — à changer pour votre projet
!
```

QUESTION 1.3

La clé 'CIEL2025' a 8 caractères. Si on chiffre un mot de passe de 20 caractères avec XOR, combien de fois la clé sera-t-elle répétée ? Complétez : pour i allant de 0 à 19, la clé utilisée à la position i est `cleSecrete[ i % ____ ]`. Donnez le caractère de clé utilisé aux positions $i=0$, $i=7$, $i=8$ et $i=15$.

1.4-BIS — Amélioration : clé personnalisée au premier démarrage

PROBLÈME DE SÉCURITÉ : CLÉ EN DUR DANS LE CODE

Actuellement, `cleSecrete = 'CIEL2025'` est écrit dans le code source.

N'importe qui lisant votre code connaît la clé.

Amélioration : au tout premier démarrage (EEPROM vierge), le programme demande à l'utilisateur de créer SA propre clé, et la sauvegarde en EEPROM.

Le code source ne contient plus aucune clé en clair.

Cette amélioration nécessite que vous ayez implémenté la persistance EEPROM (Bloc 11).

Implémentez-la APRÈS avoir validé tous les blocs précédents.

```
// — PREMIÈRE UTILISATION — CRÉER LA CLÉ —
// À appeler dans setup() si chargerEEPROM() retourne false
void premierDémarrage() {
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("Premier demarr.");
    lcd.setCursor(0, 1); lcd.print("Creez votre cle");
    delay(2000);

    // Demander la clé deux fois pour confirmer (éviter les fautes de frappe)
    char cle1[20], cle2[20];
    do {
```

```

    lcd.clear(); lcd.setCursor(0, 0); lcd.print("Nouvelle cle:");
    lcd.setCursor(0, 1);
    saisirTexte(cle1, sizeof(cle1), true);    // saisie masquée

    lcd.clear(); lcd.setCursor(0, 0); lcd.print("Confirmer:");
    lcd.setCursor(0, 1);
    saisirTexte(cle2, sizeof(cle2), true);

    if (strcmp(cle1, cle2) != 0) {
        lcd.clear(); lcd.print("Cles differentes");
        signalErreur(); delay(2000);
    }
    } while (strcmp(cle1, cle2) != 0); // recommencer si les deux saisies
diffèrent

    // Sauvegarder la clé en EEPROM (adresse 360 = après le coffre)
    EEPROM.put(360, cle1);
    memset(cle1, 0, sizeof(cle1)); // effacer la RAM
    memset(cle2, 0, sizeof(cle2));

    lcd.clear(); lcd.print("Cle sauvegardee!");
    signalOK(); delay(2000);
}

// Dans setup() - charger la clé si l'EEPROM est valide :
// if (chargerEEPROM()) {
//     EEPROM.get(360, cleSecrete); // recharger la clé personnelle
// } else {
//     premierDemarrage();
// }

```

QUESTION 1.4

Cette approche stocke la clé secrète en EEPROM à l'adresse 360. (a) Qu'est-ce qui garantit que personne ne peut lire cette clé via le Serial Monitor ? (b) Si quelqu'un a accès physique à l'Arduino, peut-il extraire la clé depuis l'EEPROM avec un programmeur externe (USBasp, AVRDUDE) ? (c) Pour une sécurité maximale, que faudrait-il faire avec la clé avant de la stocker (indice : hachage SHA-1) ?

1.5 — La fonction setup()

setup() est appelée une seule fois au démarrage. Elle initialise tous les composants.

```

// — SETUP — exécuté UNE SEULE FOIS au démarrage —————
void setup() {
    // 1. Démarrer le moniteur série pour le débogage
    Serial.begin(9600);
    Serial.println("=== Gestionnaire de mots de passe ===" );
}

```

```

// 2. Initialiser et allumer le LCD
lcd.init();
lcd.backlight();

// 3. Configurer les pins des LEDs et du buzzer en SORTIE
pinMode(LED_VERTE, OUTPUT);
pinMode(LED_BLEUE, OUTPUT);
pinMode(LED_JAUNE, OUTPUT);
pinMode(BUZZER, OUTPUT);

// 4. Test rapide des LEDs (séquence de démarrage)
digitalWrite(LED_VERTE, HIGH); delay(200);
digitalWrite(LED_VERTE, LOW);
digitalWrite(LED_BLEUE, HIGH); delay(200);
digitalWrite(LED_BLEUE, LOW);
digitalWrite(LED_JAUNE, HIGH); delay(200);
digitalWrite(LED_JAUNE, LOW);
tone(BUZZER, 1000, 100); // bip de démarrage

// 5. Message de bienvenue
lcd.setCursor(0, 0); lcd.print("Gestionnaire MDP");
lcd.setCursor(0, 1); lcd.print("  BAC PRO CIEL ");
delay(2000);
lcd.clear();
}

```

CHECKPOINT — Avant de continuer, validez chaque point :

- COMPILATION : Ctrl+R sans erreur (ignorez les warnings, pas les erreurs)
- UPLOAD : téléverser sur l'Arduino — les 3 LEDs clignotent en séquence au démarrage
- LCD : le message 'Gestionnaire MDP' s'affiche 2 secondes puis disparaît
- BUZZER : un bip court se fait entendre
- SERIAL : 'Gestionnaire de mots de passe' apparaît dans le moniteur série (9600 baud)

Signature du professeur : _____ Date : ____ / ____ / ____

BLOC 2 — SAISIE D'UN TEXTE VIA LE CLAVIER

 20 minutes

LOGIQUE DE LA SAISIE

Le clavier 4×4 ne possède que des chiffres et les lettres A, B, C, D.

Pour saisir un texte complet (nom de site, mot de passe), nous utilisons un mode simple :

- Chaque touche pressée = un caractère ajouté à la chaîne en cours
- La touche '#' = confirmer (fin de saisie)
- La touche '*' = effacer le dernier caractère (backspace)
- L'écran affiche '*' à la place des caractères (masquage du mot de passe)

La fonction saisirTexte(char* buf, int taille, bool masquer) :

Paramètre buf : le tableau char[] dans lequel écrire la saisie

Paramètre taille : la taille maximale du tableau (pour éviter le débordement)

Paramètre masquer: si true, affiche '*' à la place des vrais caractères

2.1 — Écrire la fonction saisirTexte()

Recopiez puis complétez les BLANCS (____) dans la fonction ci-dessous. Chaque blanc a une explication à droite.

```
// — SAISIE D'UN TEXTE AVEC LE CLAVIER —————
void saisirTexte(char* buf, int taille, bool masquer) {
    int longueur = 0;    // nombre de caractères saisis jusqu'ici
    char touche = 0;     // dernière touche lue
    memset(buf, 0, taille); // vider le tableau (remplir de zéros)

    digitalWrite(LED_BLEUE, HIGH); // LED bleue = saisie en cours

    while (true) {        // boucle infinie — on sort avec break
        touche = clavier.getKey(); // lire une touche (0 si aucune pressée)

        if (touche == 0) continue; // aucune touche → recommencer la boucle

        if (touche == '#') {    // '#' = fin de saisie
            break;              // sortir de la boucle while
        }
        else if (touche == '*') { // '*' = effacer le dernier caractère
            if (longueur > ____ ) { // COMPLÉTER : vérifier qu'il y a au moins 1
caractère
                longueur____;      // COMPLÉTER : décrémenter longueur (longueur-- ou
longueur-=1)
                buf[longueur] = ____; // COMPLÉTER : effacer le dernier caractère
(mettre 0)
                // Mettre à jour l'affichage LCD
                lcd.setCursor(longueur, 1);
                lcd.print(' ');      // effacer le caractère affiché
            }
        }
        else if (longueur < ____-1) { // COMPLÉTER : vérifier qu'on n'a pas dépassé
la taille max
            // Ajouter le caractère dans le tableau
            buf[____] = touche;      // COMPLÉTER : stocker touche à l'index longueur
            // Afficher sur le LCD (ligne 1, colonne = longueur)
            lcd.setCursor(____, 1); // COMPLÉTER : positionner le curseur
```

```

    if (masquer)  lcd.print('*');      // afficher '*' si masqué
    else          lcd.print(touche);   // sinon afficher le vrai caractère
    longueur____;      // COMPLÉTER : incrémenter longueur
  }
} // fin while

digitalWrite(LED_BLEUE, LOW); // éteindre la LED bleue
// buf[] contient maintenant le texte saisi, terminé par un octet nul (grâce à
memset)
}

```

LES BLANCS EN JAUNE

Chaque ligne en jaune contient un BLANC à compléter. Réfléchissez logiquement :

longueur > ____ : avant d'effacer, vérifiez que longueur > 0 (sinon c'est déjà vide)
 longueur____ : pour reculer d'un cran, on soustrait 1 à longueur
 buff[longueur] = ____ : pour effacer le caractère, on écrit 0 (le caractère nul)
 longueur < ____-1 : on réserve la dernière place pour le terminateur nul
 buff[____] = touche : on écrit à l'index longueur (avant d'incrémenter)
 lcd.setCursor(____, 1) : le curseur doit être à la colonne 'longueur'
 longueur____ : après avoir stocké et affiché, on incrémente

QUESTION 2.1

Expliquez pourquoi la condition est 'longueur < taille - 1' et non 'longueur < taille'. Que se passerait-il si on écrivait dans buff[taille] ? Comment ce dépassement de tableau s'appelle-t-il en informatique, et quelles conséquences peut-il avoir sur un système embarqué ?

2.2 — Tester la saisie

Pour tester saisirTexte() sans attendre les autres blocs, ajoutez provisoirement ce code dans loop() :

```

// TEST BLOC 2 — à mettre dans loop() provisoirement (à supprimer après test)
void loop() {
  char test[20];
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Tapez + # pour OK");
  saisirTexte(test, sizeof(test), false); // false = afficher les vrais
  caractères
  lcd.clear();
  lcd.setCursor(0, 0);  lcd.print("Vous avez tape:");
  lcd.setCursor(0, 1);  lcd.print(test);
  Serial.println(test); // vérification dans le moniteur série
  delay(3000);
}

```

CHECKPOINT — Avant de continuer, validez chaque point :

- Saisir '1', '2', '3' puis '#' → le LCD affiche '123' en ligne 2
- Tester '*' (backspace) → le dernier caractère s'efface
- Tester la limite : saisir 19 caractères → le 20ème est refusé
- Vérifier dans le Serial Monitor : la saisie complète s'affiche correctement
- Tester saisirTexte avec masquer=true : remplacer false par true → les '*' s'affichent

Signature du professeur : _____ Date : ____ / ____ / ____

2.1-BIS — Amélioration : afficher le compteur de caractères

Une fois la version de base validée, ajoutez cette amélioration UX : afficher le nombre de caractères saisis sur la ligne 0 du LCD (ex: '8/19').

```
// Ajout dans la boucle while de saisirTexte(), après avoir modifié longueur :
// Afficher le compteur en position (12, 0) - coin droit de la ligne 0
    lcd.setCursor(12, 0);
    if (longueur < 10) lcd.print(' '); // alignement si < 10
    lcd.print(longueur);
    lcd.print('/');
    lcd.print(taille - 1); // taille-1 car on réserve le terminateur
// Résultat à l'écran : le coin en haut à droite affiche ex. ' 5/19'
// Mettre à jour ce compteur à la fois lors d'un ajout ET lors d'un backspace
```

PIÈGE : NE PAS ÉCRASER LE PROMPT DE LA LIGNE 0

Le prompt (ex. 'Nom du site:') est en ligne 0.

Si vous affichez le compteur à setCursor(12, 0), vous pouvez écraser les caractères du prompt si ce dernier dépasse 12 caractères.

Raccourcissez le prompt à 12 caractères maximum : 'Nom du site:' fait exactement 12.

BLOC 3 — LE CHIFFREMENT XOR

 30 minutes

RAPPEL — COMMENT FONCTIONNE LE XOR

XOR ($\oplus$) est une opération bit à bit : $0 \oplus 1 = 1$ | $1 \oplus 0 = 1$ | $0 \oplus 0 = 0$ | $1 \oplus 1 = 0$

Pour chiffrer un message M avec une clé K :

Chiffré C = $M \oplus K$

Pour déchiffrer C avec la même clé K (propriété d'involution) :

$C \oplus K = (M \oplus K) \oplus K = M \oplus (K \oplus K) = M \oplus 0 = M$

La MÊME fonction chiffre et déchiffre.

Si la clé est plus courte que le message, on la répète cycliquement :

Position i du message $\oplus$ Position (i % longueur_clé) de la clé

Exemple : clé = 'AB' (2 chars)

$M[0] \oplus K[0\%2] = M[0] \oplus 'A'$

$M[1] \oplus K[1\%2] = M[1] \oplus 'B'$

$M[2] \oplus K[2\%2] = M[2] \oplus 'A'$ ← la clé repart depuis le début

$M[3] \oplus K[3\%2] = M[3] \oplus 'B'$ ← ...

3.1 — Vérification préalable : calculer à la main

Avant d'écrire le code, vérifiez que vous maîtrisez le calcul. Complétez ce tableau entièrement.

i	M[i]	ASCII M	K[i%4]	ASCII K	M XOR K (binaire)	Résultat	Hex
0	M	77	C	67	0100 1101 XOR 0100 0011	0000 1110 = 14	0x0E
1	D	68	I	73			
2	P	80	E	69			
3	!	33	L	76			

Message : 'MDP!' (4 chars) | Clé : 'CIEL' (4 chars)

Texte chiffré (hex) : 0x0E ____

Vérification du déchiffrement — appliquez XOR à nouveau avec 'CIEL' sur vos résultats :

3.2 — Écrire la fonction chiffrerXOR()

Complétez les blancs. Cette fonction modifie le tableau msg[] directement (en place).

```
// — CHIFFREMENT / DÉCHIFFREMENT XOR —  
// ATTENTION : la MÊME fonction chiffre ET déchiffre (propriété d'involution)
```

```
// msg : tableau de caractères à chiffrer (modifié EN PLACE)
// cle : la clé secrète (non modifiée)
void chiffrerXOR(char* msg, char* cle) {
    int n = strlen(____); // COMPLÉTER : longueur du message à chiffrer
    int k = strlen(____); // COMPLÉTER : longueur de la clé

    // Cas de sécurité : si la clé est vide, ne rien faire
    if (k == 0) return;

    for (int i = 0; ____ ; ____ ) { // COMPLÉTER : parcourir tous les octets du message
        msg[i] = msg[i] ____ cle[i % k]; // COMPLÉTER : XOR avec le bon caractère de clé
    }
    // Après la boucle : msg[] contient le texte chiffré (ou déchiffré)
    // Le terminateur nul msg[n] n'est PAS modifié (strlen() ne le compte pas)
}
```

QUESTION 3.1

Dans la boucle for, l'opérateur '^' en C correspond au XOR bit à bit. Ne confondez pas avec '^' (qui n'existe pas) ni avec '^' dans le sens mathématique 'puissance'. Cherchez dans la documentation Arduino quels sont les 6 opérateurs bit à bit disponibles en C et écrivez-les avec leur nom et leur symbole. Lequel utiliseriez-vous pour mettre à 1 le bit 3 d'un octet sans modifier les autres bits ?

💡 SI VOUS ÊTES BLOQUÉ sur chiffrerXOR()

Vérifiez ces 4 points dans l'ordre :

1. strlen(msg) → vous passez bien 'msg' et non la valeur de msg
2. strlen(cle) → vous passez bien 'cle'
3. La condition de boucle : i < n (et non i <= n qui dépasserait le tableau)
4. L'opérateur ^ en C est le XOR bit à bit. Ne pas confondre avec && (ET logique)

Si le déchiffrement ne fonctionne pas : vérifiez que vous appelez deux fois la MÊME fonction avec la MÊME clé (et non deux clés différentes).

3.3 — Tester chiffrerXOR() dans le Serial Monitor

Ajoutez provisoirement ce test dans setup() (après lcd.clear()) pour vérifier que votre implémentation est correcte :

```
// TEST BLOC 3 — à coller à la fin de setup(), avant loop()
char msg1[20] = "BONJOUR";
char cle1[20] = "CIEL2025";

Serial.print("Original : "); Serial.println(msg1);
chiffrerXOR(msg1, cle1);
```

```
Serial.print("Chiffre   : ");
for (int i = 0; i < (int)strlen(msg1); i++) {
    Serial.print((uint8_t)msg1[i], HEX); // afficher chaque octet en HEX
    Serial.print(' ');
}
Serial.println();
chiffreXOR(msg1, cle1); // deuxième appel = déchiffrement
Serial.print("Dechiffre : "); Serial.println(msg1);
// Résultat attendu : 'Dechiffre' doit afficher 'BONJOUR' (message d'origine)
```

CHECKPOINT — Avant de continuer, validez chaque point :

'Original : BONJOUR' s'affiche dans le Serial Monitor

'Chiffre : ' affiche des codes hexadécimaux (différents de BONJOUR)

'Dechiffre : BONJOUR' confirme que le déchiffrement fonctionne (involution vérifiée)

Testez aussi avec msg='AAA' et cle='AAA' → résultat chiffré = 00 00 00 (propriété A XOR A = 0)

Signature du professeur : _____ Date : ____ / ____ / ____

BLOC 4 — STOCKER UN MOT DE PASSE

 35 minutes

LOGIQUE DE LA FONCTION `stockerMdp()`

Cette fonction est le coeur du projet. Elle doit :

- Étape 1 — Vérifier qu'il reste de la place dans coffre[]
Si `nbMdp >= MAX_MDP` : afficher 'MEMOIRE PLEINE' et sortir
- Étape 2 — Demander le nom du site à l'utilisateur
Appeler `saisirTexte()` pour remplir un buffer 'site'
Si site est vide (longueur 0) : annuler et sortir
- Étape 3 — Vérifier que ce site n'existe pas déjà
Parcourir `coffre[]` et comparer les noms de sites
Si doublon trouvé : afficher 'SITE EXISTE DEJA' et sortir
- Étape 4 — Demander le mot de passe
Appeler `saisirTexte()` avec `masquer=true` (affichage en '*')
Si mot de passe vide : annuler et sortir
- Étape 5 — Chiffrer le mot de passe avec `chiffrerXOR()`
Appeler `chiffrerXOR(bufMdp, cleSecrete)`
- Étape 6 — Sauvegarder dans `coffre[nbMdp]`
Copier le nom du site dans `coffre[nbMdp].site`
Copier le mot de passe chiffré dans `coffre[nbMdp].mdpChiffre`
Incrémenter `nbMdp`
- Étape 7 — Confirmer : LED verte + bip + message 'SAUVEGARDE OK'

4.1 — Fonctions utilitaires nécessaires à `stockerMdp()`

Écrivez d'abord ces 2 petites fonctions d'aide. Elles seront utilisées dans les blocs suivants.

Fonction `signalOK()` — Retour visuel et sonore pour une action réussie

```
// Feedback positif : LED verte + bip court
void signalOK() {
    digitalWrite(LED_VERTE, HIGH);
    tone(BUZZER, 1200, 150); // fréquence 1200 Hz, durée 150 ms
    delay(300);
    digitalWrite(LED_VERTE, LOW);
}
```

Fonction `signalErreur()` — Retour visuel et sonore pour une erreur

```
// Feedback négatif : bip long grave
void signalErreur() {
    tone(BUZZER, 300, 700); // fréquence 300 Hz, durée 700 ms
    delay(800);
}
```

Fonction trouverSite() — Chercher un site dans coffre[]

Cette fonction cherche un nom de site dans le tableau coffre[] et retourne son index. Complétez les blancs.

```
// Cherche 'nomSite' dans coffre[] et retourne son index (0 à nbMdp-1)
// Retourne -1 si le site n'est pas trouvé
int trouverSite(char* nomSite) {
    for (int i = 0; i < ____; i++) {        // COMPLÉTER : parcourir jusqu'à nbMdp
        // strcmp compare deux chaînes : retourne 0 si elles sont identiques
        if (strcmp(coffre[i].site, ____)) == 0) { // COMPLÉTER : comparer avec
nomSite
            return ____; // COMPLÉTER : retourner l'index trouvé
        }
    }
    return -1; // non trouvé
}
```

QUESTION 4.1

La fonction strcmp() retourne 0 si les deux chaînes sont identiques. Pourquoi ne peut-on pas simplement écrire 'coffre[i].site == nomSite' pour comparer deux tableaux de caractères en C ? Qu'est-ce que cet opérateur compare réellement quand il est appliqué à deux tableaux char[] ? Testez dans le Serial Monitor : que donne strcmp("GMAIL", "GMAIL") ? strcmp("GMAIL", "gmail") ? strcmp("A", "B") ?

4.2 — Écrire la fonction stockerMdp()

Complétez tous les blancs en suivant la logique décrite dans l'encadré bleu de ce bloc.

```
// — STOCKER UN MOT DE PASSE —————
void stockerMdp() {
    // — ÉTAPE 1 : Vérifier la place disponible —————
    if (nbMdp >= ____ ) { // COMPLÉTER : vérifier si coffre est plein
        lcd.clear();
        lcd.setCursor(0, 0); lcd.print("MEMOIRE PLEINE !");
        lcd.setCursor(0, 1); lcd.print(nbMdp); lcd.print(" / "); lcd.print(MAX_MDP);
        signalErreur();
        delay(2000);
        return; // quitter la fonction immédiatement
    }

    // — ÉTAPE 2 : Saisir le nom du site —————
    char bufSite[20];
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("Nom du site:");
    lcd.setCursor(0, 1); lcd.print("(puis #)");
    delay(800);
    lcd.setCursor(0, 1); lcd.print("                "); // effacer ligne 2
    lcd.setCursor(0, 1);
```

```

    saisirTexte(____, sizeof(bufSite), ____); // COMPLÉTER : buffer + ne pas
    masquer

    if (strlen(bufSite) == 0) { // annulation si saisie vide
        lcd.clear(); lcd.print("Annule.");
        delay(1500); return;
    }

    // — ÉTAPE 3 : Vérifier que le site n'existe pas déjà —————
    if (trouverSite(____) != -1) { // COMPLÉTER : chercher bufSite
        lcd.clear();
        lcd.setCursor(0, 0); lcd.print("Site existant !");
        lcd.setCursor(0, 1); lcd.print(bufSite);
        signalErreur(); delay(2000); return;
    }

    // — ÉTAPE 4 : Saisir le mot de passe (masqué) —————
    char bufMdp[50];
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("Mot de passe:");
    lcd.setCursor(0, 1);
    saisirTexte(____, sizeof(bufMdp), ____); // COMPLÉTER : buffer + masquer=true

    if (strlen(bufMdp) == 0) {
        lcd.clear(); lcd.print("Annule.");
        delay(1500); return;
    }

    // — ÉTAPE 5 : Chiffrer le mot de passe avec XOR —————
    chiffrerXOR(____, ____); // COMPLÉTER : chiffrer bufMdp avec cleSecrete
    // Après cet appel, bufMdp contient le texte CHIFFRÉ (plus lisible par un
    humain)

    // — ÉTAPE 6 : Sauvegarder dans coffre[] —————
    strcpy(coffre[nbMdp].site, ____); // COMPLÉTER : copier le nom du site
    strcpy(coffre[nbMdp].mdpChiffre, ____); // COMPLÉTER : copier le mdp chiffré
    ____++; // COMPLÉTER : incrémenter le compteur de mots de passe

    // — ÉTAPE 7 : Confirmation —————
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("Sauvegarde OK !");
    lcd.setCursor(0, 1); lcd.print(bufSite);
    signalOK();
    delay(2000);
    lcd.clear();
}

```

QUESTION 4.2

Après l'appel `chiffrerXOR(bufMdp, cleSecrete)` à l'étape 5, que contient exactement `bufMdp` ? Si un attaquant lit directement la RAM de l'Arduino à cet instant, voit-il le mot de passe en clair ou chiffré ? Et AVANT cet appel, le mot de passe est-il en clair ou chiffré dans `bufMdp` ? Identifiez la fenêtre de vulnérabilité.

CHECKPOINT — Avant de continuer, validez chaque point :

Compiler sans erreur

Appeler stockerMdp() depuis loop() provisoirement pour tester

Stocker un site 'GMAIL' avec le mot de passe '1234' : 'Sauvegarde OK!' s'affiche

Vérifier dans le Serial Monitor que coffre[0].site = 'GMAIL'

Vérifier que coffre[0].mdpChiffre est bien du texte chiffré (non lisible)

Tenter de stocker à nouveau 'GMAIL' → 'Site existant!' s'affiche

Tenter de stocker sans saisie (# directement) → 'Annule.' s'affiche

Signature du professeur : _____ Date : ____ / ____ / ____

💡 SI VOUS ÊTES BLOQUÉ sur stockerMdp()

Les erreurs les plus fréquentes à cette étape :

- saisirTexte(bufSite, ...) : passer le nom du tableau SANS guillemets (pas 'bufSite')
- trouverSite(bufSite) et non trouverSite('bufSite') — c'est la variable, pas une String
- chiffrerXOR(bufMdp, cleSecrete) : les deux sont des char[], pas des String
- strcpy(coffre[nbMdp].site, bufSite) : dans cet ordre ! destination d'abord, source ensuite
- nbMdp++ DOIT être APRÈS le strcpy, jamais avant (sinon on écrit au mauvais index)

BLOC 5 — LIRE UN MOT DE PASSE

 25 minutes

LOGIQUE DE LA FONCTION lireMdp()

Cette fonction récupère et déchiffre un mot de passe. Elle doit :

- Étape 1 — Vérifier que le coffre n'est pas vide (`nbMdp == 0`)
- Étape 2 — Demander le nom du site à l'utilisateur (`saisirTexte`)
- Étape 3 — Chercher ce site dans `coffre[]` avec `trouverSite()`
 - Si non trouvé : afficher 'SITE INCONNU' et quitter
- Étape 4 — Copier le mot de passe CHIFFRÉ dans un buffer temporaire
 - IMPORTANT : copier dans un nouveau buffer — ne pas modifier `coffre[]` !
 - car on veut garder la version chiffrée pour les prochaines lectures
- Étape 5 — Déchiffrer le buffer temporaire avec `chiffrerXOR()`
 - Rappel : XOR deux fois = retour au clair (involution)
- Étape 6 — Afficher le mot de passe déchiffré sur le LCD
 - Attendre une touche pour fermer l'affichage (sécurité)
- Étape 7 — Effacer le buffer temporaire de la RAM (sécurité)

```
// — LIRE UN MOT DE PASSE —————
void lireMdp() {
    // — ÉTAPE 1 : Coffre vide ? —————
    if (nbMdp == 0) {
        lcd.clear(); lcd.print("Coffre vide !");
        signalErreur(); delay(2000); return;
    }

    // — ÉTAPE 2 : Saisir le nom du site —————
    char bufSite[20];
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("Quel site ?");
    lcd.setCursor(0, 1);
    saisirTexte(bufSite, sizeof(bufSite), false);

    // — ÉTAPE 3 : Chercher dans coffre[] —————
    int idx = trouverSite(____); // COMPLÉTER
    if (idx == -1) {
        lcd.clear();
        lcd.setCursor(0, 0); lcd.print("Site inconnu !");
        lcd.setCursor(0, 1); lcd.print(bufSite);
        signalErreur(); delay(2000); return;
    }

    // — ÉTAPE 4 : Copier le mdp chiffré dans un buffer temporaire —
    char bufTemp[50];
    strcpy(bufTemp, coffre[idx].mdpChiffre); // COMPLÉTER : utiliser l'index idx
    // bufTemp contient maintenant une COPIE du mdp chiffré
    // coffre[idx].mdpChiffre reste INTACT (toujours chiffré pour les prochaines lectures)

    // — ÉTAPE 5 : Déchiffrer bufTemp (XOR deux fois = retour au clair) —
    chiffrerXOR(____, ____); // COMPLÉTER : déchiffrer bufTemp avec cleSecrete
    // Après cet appel, bufTemp contient le mot de passe EN CLAIR

    // — ÉTAPE 6 : Afficher et attendre confirmation —————
    lcd.clear();
```

```

lcd.setCursor(0, 0); lcd.print(coffre[idx].site); // site en ligne 0
lcd.setCursor(0, 1); lcd.print(____); // COMPLÉTER : mdp déchiffré en ligne 1
signalOK();
Serial.print("MDP pour "); Serial.print(coffre[idx].site);
Serial.print(" : "); Serial.println(bufTemp); // debug Serial

// Attendre # pour fermer l'affichage
lcd.setCursor(15, 1); lcd.print(">"); // indicateur 'appuyez sur #'
while (clavier.getKey() != '#') { } // bloquer jusqu'à '#'

// — ÉTAPE 7 : Effacer le buffer temporaire de la RAM —
memset(bufTemp, 0, sizeof(bufTemp)); // écrire des zéros sur le mdp en clair
lcd.clear();
}

```

QUESTION 5.1

Pourquoi l'étape 4 copie-t-elle coffre[idx].mdpChiffre dans bufTemp AVANT de déchiffrer ? Que se passerait-il si on appelait directement chiffrerXOR(coffre[idx].mdpChiffre, cleSecrete) sans passer par bufTemp ? Testez mentalement : après cet appel direct, que contiendrait coffre[idx].mdpChiffre lors d'une deuxième lecture du même mot de passe ?

CHECKPOINT — Avant de continuer, validez chaque point :

- Stocker GMAIL/1234, puis appeler lireMdp()
- Saisir 'GMAIL' → le mot de passe '1234' s'affiche en ligne 2
- Relire une deuxième fois → '1234' s'affiche à nouveau (coffre intégré)
- Chercher un site inexistant 'AMAZON' → 'Site inconnu!' s'affiche
- Vérifier dans le Serial Monitor que bufTemp est bien effacé après affichage

Signature du professeur : _____ Date : ____ / ____ / ____

BLOC 6 — SUPPRIMER UN MOT DE PASSE

 25 minutes

LOGIQUE DE LA SUPPRESSION DANS UN TABLEAU

On ne peut pas 'faire un trou' dans un tableau en C.

Pour supprimer l'entrée à l'index idx, on doit :

Décaler tous les éléments APRÈS idx d'une position vers la gauche :

coffre[idx] = coffre[idx+1]

coffre[idx+1] = coffre[idx+2]

...jusqu'à la dernière entrée

Puis décrémenter nbMdp.

Exemple visuel avec 3 entrées, suppression de l'index 1 :

Avant : [GMAIL | FACEBOOK | AMAZON] nbMdp=3

Après : [GMAIL | AMAZON | -----] nbMdp=2

(L'ancienne entrée AMAZON s'est décalée en position 1)

```
// — SUPPRIMER UN MOT DE PASSE —————
void supprimerMdp() {
    if (nbMdp == 0) {
        lcd.clear(); lcd.print("Coffre vide !");
        signalErreur(); delay(2000); return;
    }

    // Saisir le nom du site à supprimer
    char bufSite[20];
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("Supprimer site:");
    lcd.setCursor(0, 1);
    saisirTexte(bufSite, sizeof(bufSite), false);

    // Chercher l'index du site
    int idx = trouverSite(bufSite);
    if (idx == -1) {
        lcd.clear(); lcd.print("Site inconnu !");
        signalErreur(); delay(2000); return;
    }

    // Confirmation avant suppression (touche A = oui, autre = annuler)
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("Suppr. " + String(bufSite) + "?");
    lcd.setCursor(0, 1); lcd.print("A=Oui  autre=Non");
    char confirmation = 0;
    while (confirmation == 0) { confirmation = clavier.getKey(); }
    if (confirmation != 'A') {
        lcd.clear(); lcd.print("Annule.");
        delay(1500); return;
    }

    // Décaler les entrées vers la gauche pour combler le trou
    for (int i = ____; i < ____; i++) { // COMPLÉTER : de idx à nbMdp-2 inclus
        coffre[i] = coffre[____];      // COMPLÉTER : copier l'entrée suivante
    }
}
```

```

// Effacer la dernière entrée (qui est maintenant un doublon)
memset(coffre[____].site, 0, sizeof(coffre[0].site)); // COMPLÉTER
memset(coffre[____].mdpChiffre, 0, sizeof(coffre[0].mdpChiffre)); // COMPLÉTER
// Note : coffre[nbMdp-1] est la dernière entrée avant décrémentation

____--; // COMPLÉTER : décrémenter nbMdp

// Confirmation de suppression
lcd.clear();
lcd.setCursor(0, 0); lcd.print("Supprime !");
lcd.setCursor(0, 1); lcd.print(bufSite);
digitalWrite(LED_JAUNE, HIGH);
tone(BUZZER, 600, 300);
delay(2000);
digitalWrite(LED_JAUNE, LOW);
lcd.clear();
}

```

QUESTION 6.1

Après la boucle de décalage, on appelle memset sur coffre[nbMdp-1] AVANT de décrémenter nbMdp. Expliquez pourquoi : que contient coffre[nbMdp-1] à ce moment précis, et pourquoi est-il important de l'effacer ? Que se passerait-il en termes de sécurité si on oubliait ce memset ?

CHECKPOINT — Avant de continuer, validez chaque point :

- Stocker GMAIL, FACEBOOK, AMAZON (dans cet ordre)
- Supprimer FACEBOOK → confirmer avec 'A'
- Lister les sites : GMAIL et AMAZON doivent apparaître, dans cet ordre
- AMAZON doit maintenant être à l'index 1 (était à l'index 2 avant)
- Tenter de lire FACEBOOK → 'Site inconnu!' (bien supprimé)

Signature du professeur : _____ Date : ____ / ____ / ____

💡 SI VOUS ÊTES BLOQUÉ sur supprimerMdp()

Le décalage du tableau est souvent mal compris. Voici la logique :

Avant suppression de l'index 1 : coffre[0]=A | coffre[1]=B | coffre[2]=C nbMdp=3

Boucle for i=1 : coffre[1] = coffre[2] → coffre[0]=A | coffre[1]=C | coffre[2]=C

memset sur coffre[2] → coffre[0]=A | coffre[1]=C | coffre[2]=vide

nbMdp-- → nbMdp=2

Pour la boucle : for(int i=idx; i < nbMdp-1; i++) coffre[i] = coffre[i+1];

Le memset s'applique sur coffre[nbMdp-1] AVANT de décrémenter nbMdp.

BLOC 7 — AFFICHER LA LISTE DES SITES

 15 minutes

Le LCD affiche 16 caractères sur 2 lignes. Avec un affichage cyclique, on peut montrer jusqu'à MAX_MDP sites.

```
// — AFFICHER LA LISTE DES SITES STOCKÉS —————
void afficherListe() {
    if (nbMdp == 0) {
        lcd.clear(); lcd.print("Aucun site.");
        delay(2000); return;
    }

    for (int i = 0; i < ____; i++) {    // COMPLÉTER : parcourir jusqu'à nbMdp
        lcd.clear();
        // Ligne 0 : numéro et nom du site
        lcd.setCursor(0, 0);
        lcd.print(i + 1); lcd.print(". "); lcd.print(coffre[____].site); //
        COMPLÉTER
        // Ligne 1 : indication de navigation
        lcd.setCursor(0, 1);
        if (i < nbMdp - 1) lcd.print("#=suivant");
        else                lcd.print("#=quitter");
        while (clavier.getKey() != '#') { }    // attendre '#' pour avancer
    }
    lcd.clear();
}
```

CHECKPOINT — Avant de continuer, validez chaque point :

- Avec 3 sites stockés : la liste s'affiche un par un en appuyant sur '#'
- L'ordre correspond à l'ordre de stockage
- Après le dernier site : '#=quitter' puis retour au menu
- Avec coffre vide : 'Aucun site.' s'affiche

Signature du professeur : _____ Date : ____ / ____ / ____

BLOC 8 — AUTHENTIFICATION AU DÉMARRAGE

 20 minutes

LOGIQUE DE L'AUTHENTIFICATION

Au démarrage, le programme doit vérifier l'identité de l'utilisateur.
Il demande la clé secrète, compare avec cleSecrete.

Sécurité :

- Maximum 3 tentatives avant blocage total
- En cas de blocage : while(true) { } = boucle infinie → seul un reset hardware débloque
- La clé saisie est immédiatement effacée de la RAM après vérification

IMPORTANT : cette fonction retourne true si authentifié, false sinon.
Elle est appelée UNE SEULE FOIS dans setup(), après l'initialisation.

```
// — AUTHENTIFICATION —————
bool authentifier() {
    const int MAX_TENTATIVES = 3;

    for (int tentative = 1; tentative <= ____; tentative++) { // COMPLÉTER
        // Afficher le numéro de tentative
        lcd.clear();
        lcd.setCursor(0, 0); lcd.print("Cle secrete (");
        lcd.print(tentative); lcd.print("/"); lcd.print(MAX_TENTATIVES);
        lcd.print(")");
        lcd.setCursor(0, 1);

        // Saisir la clé candidate (masquée)
        char saisieCle[20];
        saisirTexte(saisieCle, sizeof(saisieCle), ____); // COMPLÉTER : masquer=true

        // Comparer avec la clé secrète
        if (strcmp(saisieCle, ____) == 0) { // COMPLÉTER : comparer avec cleSecrete
            // Clé correcte
            memset(saisieCle, 0, sizeof(saisieCle)); // effacer la saisie
            nbEchecs = 0;
            lcd.clear(); lcd.print("Acces autorise !");
            signalOK(); signalOK(); // double bip = succès
            delay(1500); lcd.clear();
            return true; // authentication réussie
        } else {
            // Clé incorrecte
            memset(saisieCle, 0, sizeof(saisieCle)); // effacer même si incorrecte
            nbEchecs++;
            lcd.clear();
            lcd.setCursor(0, 0); lcd.print("Cle incorrecte !");
            lcd.setCursor(0, 1); lcd.print(MAX_TENTATIVES - tentative); lcd.print("
essai(s) rest.");
            signalErreur();
            delay(2000);
        }
    } // fin for

    // Trop d'échecs : BLOCAGE TOTAL
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("SYSTEME BLOQUE !");
    lcd.setCursor(0, 1); lcd.print("Reset requis.");
}
```

```
// Alarme sonore
for (int i = 0; i < 5; i++) {
    tone(BUZZER, 200, 500); delay(600);
}
while (true) { } // boucle infinie – seul un reset hardware débloque
return false;    // jamais atteint (mais requis par le compilateur)
}
```

Appelez authentifier() à la fin de setup() :

```
// À ajouter À LA FIN de setup(), juste avant la fermeture de l'accolade
if (!authentifier()) {
    while (true) { } // sécurité supplémentaire si authentifier() retourne
false
}
```

QUESTION 8.1

Analysez la sécurité de cette authentification : (a) Pourquoi appelle-t-on memset(saisieCle, 0, ...) même quand la clé est incorrecte ? (b) La boucle while(true) après 3 échecs bloque-t-elle le watchdog timer de l'Arduino ? Cherchez ce qu'est le watchdog timer et si cela peut poser un problème. (c) Quelle amélioration de sécurité apporterait un délai CROISSANT entre les tentatives (1s, 4s, 16s) plutôt qu'un délai fixe ?

CHECKPOINT — Avant de continuer, validez chaque point :

- Saisir la mauvaise clé → 'Cle incorrecte!' + le nombre de tentatives restantes
- 3 mauvaises clés → 'SYSTEME BLOQUE!' + alarme buzzer
- Seul le bouton RESET de l'Arduino débloque le système
- Saisir la bonne clé 'CIEL2025' → 'Acces autorise!' + 2 bips + accès au menu

Signature du professeur : _____ Date : ____ / ____ / ____

BLOC 9 — LE MENU PRINCIPAL DANS LOOP()

 20 minutes

LES 3 MODES DU GESTIONNAIRE (conformes au cahier des charges)

Le programme doit implémenter exactement ces 3 modes :

MODE 1 — STOCKER (touche 1)

Saisir nom du site → saisir mot de passe → CHIFFRER avec clé → stocker
LED verte + bip sonore en confirmation

MODE 2 — RÉCUPÉRER (touche 2)

Saisir nom du site → trouver → DÉCHIFFRER → afficher 10 secondes sur LCD
LED BLEUE allumée pendant l'affichage → extinction automatique après 10 s

MODE 3 — CHANGER LA CLÉ SECRÈTE (touche B)

Vérifier l'ancienne clé → saisir nouvelle clé → DÉCHIFFRER TOUS les MDP
→ RECHIFFRER TOUS avec la nouvelle clé → LED jaune + mise à jour sécurisée

Touche 3 — Supprimer un site

Touche 4 — Lister les sites stockés

9.0 — Correction du MODE 2 : timeout 10 secondes + LED bleue

Le cahier des charges précise : affichage TEMPORAIRE (10 secondes) avec LED bleue. Voici la version corrigée de lireMdp() pour les étapes 6 et 7 :

```
// — ÉTAPE 6 CORRIGÉE : Afficher avec timeout 10 secondes ————
lcd.clear();
lcd.setCursor(0, 0); lcd.print(coffre[idx].site);
lcd.setCursor(0, 1); lcd.print(bufTemp);          // mot de passe en clair
digitalWrite(LED_BLEUE, HIGH);                    // LED bleue ON pendant
l'affichage
signalOK();

// Attendre 10 secondes OU que l'utilisateur appuie sur # pour fermer plus tôt
unsigned long debut = millis();                    // noter l'heure de début (en millisecondes)
while (millis() - debut < 10000) { // boucler pendant 10 000 ms = 10 secondes
    // Afficher le compte à rebours en coin
    int restant = 10 - (int)((millis() - debut) / 1000);
    lcd.setCursor(14, 0);
    if (restant < 10) lcd.print(' '); // espace si 1 chiffre
    lcd.print(restant);                // ex: '9', '8', ..., '1'
    if (clavier.getKey() == '#') break; // '#' = fermer immédiatement
    delay(200);                        // rafraîchir toutes les 200 ms
}

// — ÉTAPE 7 CORRIGÉE : Effacer affichage + éteindre LED ————
digitalWrite(LED_BLEUE, LOW);           // LED bleue OFF
memset(bufTemp, 0, sizeof(bufTemp));    // effacer le MDP en clair de la RAM
lcd.clear();
```

QUESTION 9.0

La fonction millis() retourne le nombre de millisecondes depuis le démarrage de l'Arduino. (a) Pourquoi utilise-t-on millis() plutôt qu'un simple delay(10000) pour le compte à rebours ? (b) Que se passerait-il

avec `delay(10000)` si l'utilisateur appuie sur '#' pendant l'attente ? (c) Après environ 49 jours de fonctionnement, `millis()` déborde (overflow sur `uint32_t`). Expliquez pourquoi l'expression `millis() - debut` reste correcte même après ce débordement (indice : arithmétique modulo 2^{32}).

BLOC 8-BIS — MODE 3 : CHANGER LA CLÉ SECRÈTE

🕒 35 minutes

POURQUOI CE MODE EST CRITIQUE ET COMPLEXE

Changer la clé secrète ne se résume pas à modifier la variable `cleSecrete`.

PROBLÈME : tous les mots de passe stockés sont chiffrés avec l'ANCIENNE clé.

Si on change `cleSecrete` directement, toutes les données deviennent illisibles.

La solution en 3 temps :

1. Déchiffrer chaque MDP avec l'ANCIENNE clé → on obtient le MDP en clair
2. Rechiffrer immédiatement avec la NOUVELLE clé → on obtient le nouveau chiffré
3. Écraser l'entrée dans `coffre[]` avec le nouveau chiffré

Ce processus s'appelle une RE-ENCRYPTION (ou re-chiffrement à la volée).

Il est utilisé dans les vrais gestionnaires de mots de passe quand on change son mot de passe maître.

RISQUE : si le programme est interrompu (coupure de courant) entre l'étape 1 et 3, certains MDP seraient à moitié re-chiffrés. En production, on ferait d'abord une copie de sauvegarde. Ici on suppose que la connexion reste stable.

8-BIS.1 — Schéma de la re-encryption

```
coffre[0].mdpChiffre = 'P'(80) XOR 'C'(67) = 19 | 'a'(97) XOR 'I'(73) = 56 | ...  
                ↑ chiffré avec ancienne clé 'CIEL2025'
```

Étape 1 – déchiffrer avec ANCIENNE clé :

```
bufTemp[0] = 19 XOR 'C'(67) = 'P'(80) ← on retrouve le MDP en clair  
bufTemp[1] = 56 XOR 'I'(73) = 'a'(97)  
... bufTemp = 'Pass1234' (en clair en RAM – zone de vulnérabilité !)
```

Étape 2 – rechiffrer immédiatement avec NOUVELLE clé 'ARDUINO2025' :

```
bufTemp[0] = 'P'(80) XOR 'A'(65) = 17  
bufTemp[1] = 'a'(97) XOR 'R'(82) = 51  
... bufTemp = nouveau chiffré
```

Étape 3 – écraser `coffre[0].mdpChiffre` avec `bufTemp`

```
strcpy(coffre[0].mdpChiffre, bufTemp)
```

Répéter pour `coffre[1]`, `coffre[2]`, ... `coffre[nbMdp-1]`

Étape finale – mettre à jour `cleSecrete` avec la nouvelle clé
`strncpy(cleSecrete, nouvelleCle, sizeof(cleSecrete) - 1)`

8-BIS.2 — Écrire la fonction `changerCle()`

Complétez tous les blancs. Suivez scrupuleusement l'ordre des opérations pour ne corrompre aucun mot de passe.

```
// — MODE 3 : CHANGER LA CLÉ SECRÈTE —————  
// Déchiffre tous les MDP avec l'ancienne clé, rechiffre avec la nouvelle.  
void changerCle() {  
    // — ÉTAPE 1 : Vérifier l'ancienne clé (authentification) —————
```

```

lcd.clear();
lcd.setCursor(0, 0); lcd.print("Ancienne cle:");
lcd.setCursor(0, 1);
char anciCle[20];
saisirTexte(anciCle, sizeof(anciCle), true); // saisie masquée

if (strcmp(anciCle, ____ ) != 0) { // COMPLÉTER : comparer avec cleSecrete
    lcd.clear(); lcd.print("Cle incorrecte!");
    memset(anciCle, 0, sizeof(anciCle));
    signalErreur(); delay(2000); return;
}

// — ÉTAPE 2 : Saisir et confirmer la nouvelle clé —————
char nouvCle[20], confirmCle[20];
do {
    lcd.clear(); lcd.setCursor(0, 0); lcd.print("Nouvelle cle:");
    lcd.setCursor(0, 1);
    saisirTexte(____, sizeof(nouvCle), true); // COMPLÉTER : saisir nouvCle

    lcd.clear(); lcd.setCursor(0, 0); lcd.print("Confirmer cle:");
    lcd.setCursor(0, 1);
    saisirTexte(____, sizeof(confirmCle), true); // COMPLÉTER : saisir confirmCle

    if (strcmp(nouvCle, confirmCle) != 0) {
        lcd.clear(); lcd.print("Cles diff. !");
        signalErreur(); delay(2000);
    }
} while (strcmp(nouvCle, confirmCle) != 0); // recommencer si les saisies
diffèrent

// Vérification : la nouvelle clé ne doit pas être vide
if (strlen(nouvCle) == 0) {
    lcd.clear(); lcd.print("Cle vide !");
    signalErreur(); delay(2000); return;
}

// — ÉTAPE 3 : RE-ENCRYPTION de tous les mots de passe —————
lcd.clear();
lcd.setCursor(0, 0); lcd.print("Mise a jour...");
digitalWrite(LED_JAUNE, HIGH); // LED jaune = opération en cours

for (int i = 0; i < ____; i++) { // COMPLÉTER : parcourir tous les MDP
stockés
    char bufTemp[50];
    // Copier le MDP chiffré (avec ancienne clé) dans bufTemp
    strcpy(bufTemp, coffre[i].____); // COMPLÉTER : champ mdpChiffre

    // Déchiffrer bufTemp avec l'ANCIENNE clé → bufTemp contient le MDP en clair
    chifftrerXOR(____, ____); // COMPLÉTER : bufTemp + anciCle

    // Rechiffrer immédiatement bufTemp avec la NOUVELLE clé
    chifftrerXOR(____, ____); // COMPLÉTER : bufTemp + nouvCle

    // Écraser l'entrée du coffre avec le nouveau chiffré
    strcpy(coffre[i].mdpChiffre, ____); // COMPLÉTER : copier bufTemp

    // Effacer bufTemp (ne pas laisser le MDP en clair en RAM)
    memset(____, 0, sizeof(bufTemp)); // COMPLÉTER

    // Afficher la progression sur le LCD (ex: '3/5')
    lcd.setCursor(0, 1);
    lcd.print(i + 1); lcd.print("/"); lcd.print(nbMdp);
}

```

```

}

// — ÉTAPE 4 : Mettre à jour cleSecrete avec la nouvelle clé —
// strncpy copie au maximum sizeof(cleSecrete)-1 caractères et garantit le
// terminateur
strncpy(cleSecrete, ____, sizeof(cleSecrete) - 1); // COMPLÉTER : copier
nouvCle
cleSecrete[sizeof(cleSecrete) - 1] = 0; // forcer le terminateur nul

// — ÉTAPE 5 : Effacer toutes les variables sensibles de la RAM —
memset(anciCle, 0, sizeof(anciCle));
memset(nouvCle, 0, sizeof(nouvCle));
memset(confirmCle, 0, sizeof(confirmCle));

// — ÉTAPE 6 : Confirmation —
digitalWrite(LED_JAUNE, LOW);
lcd.clear();
lcd.setCursor(0, 0); lcd.print("Cle changee !");
lcd.setCursor(0, 1); lcd.print(nbMdp); lcd.print(" MDP mis a jour");
tone(BUZZER, 800, 100); delay(150);
tone(BUZZER, 1000, 100); delay(150);
tone(BUZZER, 1200, 200);
delay(3000);
lcd.clear();
}

```

ORDRE IMPÉRATIF DANS LA BOUCLE

Chaque itération DOIT suivre cet ordre exact, sans exception :

1. strcpy(bufTemp, coffre[i].mdpChiffre) ← copier le chiffré
2. chiffrierXOR(bufTemp, anciCle) ← déchiffrer → clair en RAM
3. chiffrierXOR(bufTemp, nouvCle) ← rechiffrer → nouveau chiffré
4. strcpy(coffre[i].mdpChiffre, bufTemp) ← mettre à jour le coffre
5. memset(bufTemp, 0, sizeof(bufTemp)) ← effacer le clair de la RAM

Si on fait étape 4 avant étape 3, on écrase le coffre avec le MDP EN CLAIR !

Si on oublie étape 5, le MDP en clair reste en RAM jusqu'à la prochaine itération.

QUESTION 8-BIS.1

Analysez la fenêtre de vulnérabilité lors de la re-encryption : (a) Entre l'étape 2 et l'étape 3 de la boucle, le MDP est en clair dans bufTemp (variable locale sur la pile). Combien de temps (approximativement en µs ou ms) cette fenêtre est-elle ouverte pour un seul MDP ? (b) Si un attaquant peut lire la RAM de l'Arduino en temps réel (ex: via JTAG), peut-il voir le MDP pendant cette fenêtre ? (c) Pourquoi la variable bufTemp est-elle déclarée DANS la boucle for et non avant ? Quelle différence cela fait-il pour la sécurité ?

QUESTION 8-BIS.2

La re-encryption traite les MDP UN PAR UN. Tracez l'exécution complète pour 2 mots de passe stockés (GMAIL/Pass1, AMAZON/Secret9) en changeant la clé de 'CIEL' à 'NOUVEAU'. Pour chaque itération, indiquez la valeur de bufTemp après l'étape 2 et après l'étape 3. Vérifiez qu'après la boucle, lireMdp('GMAIL') avec la nouvelle clé 'NOUVEAU' retourne bien 'Pass1'.

CHECKPOINT — Avant de continuer, validez chaque point :

Stocker GMAIL/Pass1 et AMAZON/Secret9 avec la clé CIEL2025
Lire GMAIL → 'Pass1' s'affiche (vérification avant changement)
Appuyer sur B → saisir ancienne clé 'CIEL2025' → saisir nouvelle clé 'NOUVEAU25' × 2
Progression : '1/2' puis '2/2' s'affichent → 'Cle changee !' + mélodie 3 notes
Lire GMAIL avec la nouvelle clé → 'Pass1' s'affiche encore (re-encryption réussie)
Tenter de lire GMAIL en changeant cleSecrete manuellement retour à 'CIEL2025' → charabia (preuve que le coffre a bien été rechiffré)

Signature du professeur : _____ Date : ____ / ____ / ____

9.1 — Le menu loop() mis à jour avec le MODE 3

Mettez à jour loop() pour afficher les 5 options et router vers changerCle() sur la touche B.

```
// — MENU PRINCIPAL MIS À JOUR — 5 options —————
void loop() {
    lcd.clear();
    // Ligne 0 : options 1 à 4
    lcd.setCursor(0, 0); lcd.print("1Stk 2Lir 3Sup 4Lst");
    // Ligne 1 : option B + nombre de MDP stockés
    lcd.setCursor(0, 1); lcd.print("B=CleSecrete ");
    lcd.print(nbMdp); lcd.print("/"); lcd.print(MAX_MDP);

    char choix = 0;
    while (choix == 0) { choix = clavier.getKey(); }

    switch (choix) {
        case '1': stockerMdp();    break;    // MODE 1 : stocker
        case '2': lireMdp();       break;    // MODE 2 : récupérer (10s + LED bleue)
        case '3': supprimerMdp(); break;    // suppression
        case '4': afficherListe(); break;    // liste des sites
        case 'B': changerCle();    break;    // MODE 3 : changer la clé secrète
        default: break;
    }
}
```

```
}  
}
```

QUESTION 9.1

La structure switch/case est utilisée à la place d'une chaîne if/else if. (a) Quels sont les avantages du switch/case pour un menu ? (b) Que fait le 'default' ici ? (c) Si on oublie un 'break' après 'case 2', que se passe-t-il exactement ? Tracez l'exécution pour choix='2' sans break et expliquez le comportement.

BLOC 10 — TESTS COMPLETS ET VALIDATION FINALE

 25 minutes

Exécutez CHAQUE scénario de test dans l'ordre. Cochez si le résultat correspond à l'attendu.

#	Scénario de test	Résultat attendu	Résultat obtenu	OK ?
T1	Démarrage → saisir 'CIEL2025' + #	'Acces autorise!' + 2 bips + menu		☐
T2	Menu → '1' → Nom='GMAIL' + # → MDP='Pass1' + #	'Sauvegarde OK!' + LED verte + 1 entrée dans coffre		☐
T3	Menu → '1' → Nom='AMAZON' + # → MDP='Zap99!' + #	'Sauvegarde OK!' + 2 entrées dans coffre		☐
T4	Menu → '1' → Nom='GMAIL' + # (doublon)	'Site existant!' + signalErreur		☐
T5	Menu → '2' → Nom='GMAIL' + #	LCD ligne 0='GMAIL', ligne 1='Pass1' + LED verte		☐
T6	(Suite T5) Relire GMAIL une 2ème fois	'Pass1' s'affiche encore — coffre intact		☐
T7	Menu → '2' → Nom='TWITTER' (inexistant)	'Site inconnu!' + signalErreur		☐
T8	Menu → '4' (liste)	'1. GMAIL' puis '#' puis '2. AMAZON' puis '#'		☐
T9	Menu → '3' → Nom='GMAIL' → confirmation 'A'	'Supprime!' + LED jaune + AMAZON devient index 0		☐
T10	Menu → '2' → Nom='GMAIL' (supprimé)	'Site inconnu!' (GMAIL bien supprimé)		☐
T11	Menu → '2' → Nom='AMAZON' (doit encore marcher)	'Zap99!' s'affiche (AMAZON intact après suppression GMAIL)		☐
T12	Reset Arduino → saisir mauvaise clé 3 fois	'SYSTEME BLOQUE!' + alarme → seul reset débloque		☐
T13	Stocker 5 MDP, puis essayer d'en ajouter un 6ème	'MEMOIRE PLEINE !' + nbMdp affiché		☐
T14	Saisie avec backspace : saisir '1','2','*','3' + #	Résultat = '13' (le '2' a été effacé)		☐

10.1 — Analyse finale : vérification du chiffrement XOR

Pour confirmer que le chiffrement fonctionne correctement, effectuez cette vérification dans le Serial Monitor.

```
// À ajouter temporairement dans stockerMdp(), juste après chiffrerXOR()
// (Retirer avant de rendre le projet final)
Serial.print("Site      : "); Serial.println(bufSite);
Serial.print("MDP clair  : "); Serial.println(bufMdp_avant_xor); // buffer à
créer avant l'appel XOR
Serial.print("MDP chiffre (hex) : ");
for (int i = 0; i < (int)strlen(bufMdp); i++) {
    if ((uint8_t)bufMdp[i] < 16) Serial.print('0'); // zéro de padding
    Serial.print((uint8_t)bufMdp[i], HEX);
    Serial.print(' ');
}
Serial.println();
```

Site stocké	MDP en clair (saisi)	MDP chiffré (hex affiché)	Déchiffré = clair ?
GMAIL	1234	(à noter ici)	☐ Oui ☐ Non
TEST	AAAA	(à noter ici)	☐ Oui ☐ Non
MON SITE	MonPass2025	(à noter ici)	☐ Oui ☐ Non

COMMENT LIRE UN MESSAGE D'ERREUR ARDUINO

Le compilateur Arduino génère des messages d'erreur en anglais dans la console (en bas).
Ne paniquez pas — lisez-les méthodiquement :

Format d'un message d'erreur :

sketch.ino : LIGNE_NUMÉRO : error : DESCRIPTION_DU_PROBLÈME

La ligne indiquée est SOUVENT juste après l'erreur réelle.

Si vous ne voyez pas le problème ligne N, regardez la ligne N-1 ou N-2.

Règle d'or : corrigez TOUJOURS la PREMIÈRE erreur en premier.

Une seule erreur peut en générer 10 autres en cascade.

Message d'erreur (extrait)	Cause probable	Solution
'expected ';' before '}' token	Oubli d'un point-virgule (;) à la fin d'une ligne	Regardez la ligne AVANT celle indiquée — ajoutez le ;
'was not declared in this scope'	Variable ou fonction utilisée avant d'être déclarée	Vérifiez l'orthographe exacte (majuscules !). Déclarez avant d'utiliser.
'too many arguments to function'	Appel de fonction avec plus de paramètres qu'attendu	Comptez les paramètres dans la déclaration et dans l'appel
'invalid conversion from char to char*'	Vous passez un char là où un char* (pointeur) est attendu	Pour passer un tableau buf à une fonction char* : écrire buf (sans &)
'a function-definition is not allowed here'	Fonction définie À L'INTÉRIEUR d'une autre fonction	Fermez tous les {} de la fonction en cours avant d'en ouvrir une nouvelle
'ISO C++ forbids converting string to char*'	Passage d'une String littérale là où char* est attendu	Utilisez des guillemets doubles ET la forme correcte : chiffreurXOR(buf, "CLE")
Low memory available (< 500 bytes)	Programme trop grand pour la RAM de l'Arduino (2048 bytes)	Réduisez MAX_MDP, raccourcissez les messages lcd.print(), utilisez F()
'expected ')' before '.' token'	Parenthèse manquante dans un if() ou while()	Comptez les parenthèses ouvrantes et fermantes — elles doivent s'équilibrer

ASTUCE — ÉCONOMISER LA RAM AVEC F()

Sur Arduino, les String littérales (textes entre guillemets) sont copiées en RAM au démarrage.
La macro F() force le stockage en Flash (mémoire programme, 32 Ko) plutôt qu'en RAM.

Sans F() : lcd.print("Sauvegarde OK !"); → 16 octets en RAM

Avec F() : lcd.print(F("Sauvegarde OK !")); → 0 octet en RAM, 16 en Flash

Si le compilateur indique 'low memory available', ajoutez F() sur tous vos lcd.print().

Sur un projet complet, cela peut libérer 200 à 400 octets de RAM.

Problèmes à l'exécution (pas de compilation mais comportement incorrect)

Symptôme observé	Cause probable	Comment vérifier / corriger
LCD reste noir après l'upload	Adresse I2C incorrecte (0x27 vs 0x3F) ou contraste trop faible	Lancer I2C Scanner. Tourner le potentiomètre derrière le module LCD.
Mot de passe déchiffré différent de l'original	chiffreXOR() appelé directement sur coffre[] au lieu de bufTemp	Vérifiez que lireMdp() utilise bien strcpy(bufTemp, ...) avant le XOR
Arduino redémarre tout seul sous charge	Débordement de pile (stack overflow) ou RAM saturée	Réduisez la taille des tableaux locaux dans les fonctions
La liste affiche des sites vides ou corrompus	Suppression incorrecte — memset ou décalage manquant	Ajoutez Serial.println de coffre[i].site après chaque opération
Le clavier ne répond pas	Broches de lignes/colonnes inversées ou mauvais pinMode	Vérifiez pinsLignes[] vs pinsColonnes[] et le câblage physique

BLOC 11 (BONUS) — PERSISTANCE DES DONNÉES : LA MÉMOIRE EEPROM

 30 minutes — BONUS si le programme de base fonctionne

LE PROBLÈME ACTUEL

Votre programme fonctionne bien — mais il a une limitation critique :
Si l'Arduino est mis hors tension (coupure de courant, débranchement USB),
TOUS les mots de passe stockés sont perdus.

Pourquoi ? Parce que le tableau `coffre[]` est en RAM (mémoire volatile).
La RAM est effacée à chaque mise hors tension.

Solution : utiliser l'EEPROM de l'Arduino.

QU'EST-CE QUE L'EEPROM ?

EEPROM = Electrically Erasable Programmable Read-Only Memory

Caractéristiques de l'Arduino Uno (ATmega328P) :

- Capacité : 1024 octets (1 Ko)
- Durée de vie : 100 000 cycles d'écriture par adresse
- Conservation : 20 ans sans alimentation
- Vitesse : lecture rapide, écriture lente (3.3 ms par octet)

Comparaison des mémoires de l'Arduino :

Flash (32 Ko) — stocke le programme — effacée à chaque upload
SRAM (2 Ko) — variables d'exécution — effacée à chaque reset
EEPROM (1 Ko) — données persistantes — SURVIT aux reset/coupures

Notre `coffre[5]` de `MotDePasse` occupe environ 350 octets.
L'EEPROM de 1024 octets peut largement le stocker.

11.1 — Calcul de l'espace nécessaire

Avant d'écrire, calculez si tout tient en EEPROM.

Donnée à stocker	Taille par entrée	× MAX_MDP (5)	= Octets total
<code>coffre[i].site (char[20])</code>	20 octets	× 5	100 octets
<code>coffre[i].mdpChiffre (char[50])</code>	50 octets	× 5	250 octets
<code>nbMdp (int)</code>	2 octets	× 1	2 octets
Marqueur de validité (signature)	4 octets	× 1	4 octets
TOTAL	—	—	356 octets (sur 1024)

QUESTION 11.1

Le total de 356 octets représente quel pourcentage de l'EEPROM (1024 octets) ? Si on augmentait `MAX_MDP` à 10 et la taille du champ `site` à `char[30]` et `mdpChiffre` à `char[60]`, recalculez l'espace nécessaire. Tient-il encore dans l'EEPROM ? Et si on voulait stocker 20 mots de passe ?

11.2 — La bibliothèque EEPROM — Fonctions essentielles

```
// La bibliothèque EEPROM est incluse avec Arduino IDE — pas besoin de
l'installer
#include <EEPROM.h>      // à ajouter en tête du programme, avec les autres
#include

// Les 3 fonctions de base :

// 1. Écrire 1 octet à l'adresse 'addr'
EEPROM.write(addr, valeur_8bits);
// Exemple : EEPROM.write(0, 42); → écrit 42 à l'adresse 0

// 2. Lire 1 octet à l'adresse 'addr'
byte valeur = EEPROM.read(addr);
// Exemple : byte v = EEPROM.read(0); → lit 1'octet à l'adresse 0

// 3. Écrire un BLOC de données (n'importe quelle structure)
EEPROM.put(addr, donnee); // écrit sizeof(donnee) octets à partir de addr
// Exemple : EEPROM.put(10, coffre[0]); → écrit toute la struct MotDePasse

// 4. Lire un BLOC de données
EEPROM.get(addr, donnee); // lit sizeof(donnee) octets à partir de addr
// Exemple : EEPROM.get(10, coffre[0]); → relit la struct sauvegardée

// EEPROM.put() est plus efficace que EEPROM.write() car il vérifie si l'octet
// a déjà cette valeur avant d'écrire → préserve les cycles de vie EEPROM
```

11.3 — Plan d'adressage EEPROM

Avant d'écrire en EEPROM, on planifie EXACTEMENT où chaque donnée sera stockée.

Donnée	Adresse début	Taille	Formule de calcul	Exemple pour i=2
Signature de validité	0	4 octets	—	Adresse fixe 0
nbMdp	4	2 octets	—	Adresse fixe 4
coffre[i].site	$6 + i \times 70$	20 octets	$6 + i \times (20+50)$	$6 + 2 \times 70 = 146$
coffre[i].mdpChiffre	$26 + i \times 70$	50 octets	$6 + i \times 70 + 20$	$26 + 2 \times 70 = 166$

QUESTION 11.2

Vérifiez le plan d'adressage : l'adresse de fin de la dernière entrée (coffre[4].mdpChiffre) est $26 + 4 \times 70 + 50 = \underline{\hspace{2cm}}$. Est-ce bien inférieur à 1024 (taille de l'EEPROM) ? Expliquez pourquoi la taille d'une entrée est 70 octets (20+50) et non 69.

11.4 — Écrire les fonctions sauvegarderEEPROM() et chargerEEPROM()

```
// — CONSTANTES D'ADRESSAGE EEPROM —————
#define EEPROM_SIGNATURE 0 // adresse de la signature (4 octets)
#define EEPROM_NBMDP 4 // adresse de nbMdp (2 octets)
#define EEPROM_COFFRE 6 // adresse du premier mot de passe
#define TAILLE_ENTREE 70 // sizeof(MotDePasse) = 20 + 50
#define MAGIC_NUMBER 0x4D4450 // 'MDP' en hex – signature de validité

// — SAUVEGARDER LE COFFRE EN EEPROM —————
void sauvegarderEEPROM() {
    // Écrire la signature 'MDP' pour savoir si l'EEPROM contient des données
    // valides
    EEPROM.put(EEPROM_SIGNATURE, (uint32_t)MAGIC_NUMBER);
    // Écrire le nombre de mots de passe
    EEPROM.put(EEPROM_NBMDP, nbMdp);
    // Écrire chaque entrée du coffre
    for (int i = 0; i < nbMdp; i++) {
        int adresseEntree = EEPROM_COFFRE + (i * TAILLE_ENTREE);
        EEPROM.put(adresseEntree, coffre[i].site);
        EEPROM.put(adresseEntree + 20, coffre[i].mdpChiffre);
    }
    // Feedback
    Serial.println("EEPROM: sauvegarde OK");
}

// — CHARGER LE COFFRE DEPUIS L'EEPROM —————
// Retourne true si des données valides ont été chargées, false si EEPROM vierge
bool chargerEEPROM() {
    // Vérifier la signature – si elle est absente, l'EEPROM est vierge
    uint32_t signature;
    EEPROM.get(EEPROM_SIGNATURE, signature);
    if (signature != MAGIC_NUMBER) {
        Serial.println("EEPROM: vierge ou invalide");
        return false; // rien à charger
    }
    // COMPLÉTER : lire nbMdp depuis l'EEPROM
    EEPROM.get(____, ____); // adresse EEPROM_NBMDP, variable nbMdp

    // Sécurité : nbMdp ne peut pas dépasser MAX_MDP
    if (nbMdp > MAX_MDP || nbMdp < 0) { nbMdp = 0; return false; }

    // COMPLÉTER : lire chaque entrée du coffre
    for (int i = 0; i < ____; i++) { // parcourir jusqu'à nbMdp
        int adresseEntree = EEPROM_COFFRE + (i * TAILLE_ENTREE);
        EEPROM.get(____, coffre[i].site); // COMPLÉTER : adresse du site
        EEPROM.get(____, coffre[i].mdpChiffre); // COMPLÉTER : adresse du mdp
    }
    Serial.print("EEPROM: "); Serial.print(nbMdp); Serial.println(" mdp charges");
    return true;
}
```

11.5 — Intégrer la persistance dans le programme

Deux modifications dans le programme existant pour activer la persistance :

```
// — MODIFICATION 1 : charger au démarrage (dans setup(), APRÈS authentifier())
—
if (chargerEEPROM()) {
  lcd.clear();
  lcd.setCursor(0, 0); lcd.print("Coffre charge!");
  lcd.setCursor(0, 1); lcd.print(nbMdp); lcd.print("-mot(s)-passe");
  delay(2000); lcd.clear();
}

// — MODIFICATION 2 : sauvegarder après CHAQUE modification (stockerMdp,
supprimerMdp)
// À la fin de stockerMdp(), juste avant signalOK() :
sauvegarderEEPROM(); // ← ajouter cette ligne

// À la fin de supprimerMdp(), juste avant le feedback :
sauvegarderEEPROM(); // ← ajouter cette ligne
```

CHECKPOINT — Avant de continuer, validez chaque point :

- Stocker 2 mots de passe
- Débrancher puis rebrancher l'Arduino (coupure d'alimentation)
- Au redémarrage : 'Coffre charge ! 2 mot(s) de passe' s'affiche
- Lire les 2 mots de passe → ils sont toujours là, correctement déchiffrés
- Dans le Serial Monitor : 'EEPROM: 2 mdp charges' confirme le chargement

Signature du professeur : _____ Date : ____ / ____ / ____

QUESTION 11.3

La signature MAGIC_NUMBER (0x4D4450 = 'MDP') est stockée à l'adresse 0. (a) Que se passe-t-il si l'Arduino est utilisé pour la première fois (EEPROM vierge) ? Que lit-on à l'adresse 0 ? Pourquoi ce n'est pas 0x4D4450 ? (b) Si une coupure de courant survient PENDANT l'écriture EEPROM (entre sauvegarder le site et sauvegarder le mdpChiffre), que risque-t-il de se passer à la prochaine relecture ? Comment pourrait-on rendre cette sauvegarde plus robuste (mot clé : 'écriture atomique') ?

BILAN ET AUTO-ÉVALUATION

Compétence	Auto-évaluation	Professeur
Je complète les blancs de saisirTexte() correctement	☐ Acquis ☐ En cours	/5
Je code chiffrerXOR() — la boucle et l'opérateur ^	☐ Acquis ☐ En cours	/5
Je comprends pourquoi XOR deux fois = retour au clair	☐ Acquis ☐ En cours	/5

Compétence	Auto-évaluation	Professeur
Je code stockerMdp() — les 7 étapes dans le bon ordre	☐ Acquis ☐ En cours	/5
Je code lireMdp() — copie dans buffer temporaire avant XOR	☐ Acquis ☐ En cours	/5
Je code supprimerMdp() — décalage du tableau	☐ Acquis ☐ En cours	/5
Mon programme passe les 14 tests de la grille T1 à T14	☐ Acquis ☐ En cours	/5
[BONUS] sauvegarderEEPROM() et chargerEEPROM() fonctionnent	☐ Acquis ☐ En cours	/5 bonus
[BONUS] Les MDP survivent à une coupure de courant	☐ Acquis ☐ En cours	/5 bonus

Félicitations — Votre gestionnaire de mots de passe fonctionne !